Jain University

Department of Computer Science & Engineering (Software Engineering)

First Review Report

Design, Literature Update & Progress

ML-Based Personalised Adaptive E-Learning System

Submitted by	Anirudh Choudhary and Sushrut Tiwari
Programme	Bachelor of Technology — Software Engineering
Semester	Semester VII, Academic Year 2026–2027
Guide	_____
Date of Review	_____

1. Recap of Problem Statement and Objectives

As established at the Zeroth Review, traditional e-learning platforms are one-size-fits-all: every learner receives the same sequence, pace and difficulty regardless of prior knowledge, learning speed or weak areas. This project replaces that fixed path with one that reads each student's own activity data and adjusts what they see next.

The objective remains to design, implement and evaluate a machine-learning-based adaptive e-learning system built on the Open University Learning Analytics Dataset (OULAD), combining three stages in one continuous loop: classification of a student's outcome (Pass / Fail / Distinction / Withdrawn), clustering of similar learners, and a recommendation step that converts both into a content suggestion. The system must remain usable across all ages and education levels, and at least 80–85% of the work must be original, with only 15–20% drawn from published techniques.

2. Updated Literature Survey

Author(s) / Year	Method	Key Contribution	Limitation
Kuzilek et al., 2017	OULAD dataset	Open, anonymised dataset of student demographics, assessments and VLE clickstream across multiple courses; basis for this project's training data.	Reflects one institution's distance-learning context; outcome labels are course-level.
Breiman, 2001	Random Forest	Ensemble of decision trees voting on the outcome; robust to overfitting and gives feature importances.	Less directly interpretable than a single tree.
[Assigned paper — authors, year]	Q-learning recommender	Recommends content using Q-learning with transition probabilities between learning states.	Trained on synthetic data with hand-crafted rewards; does not generalise across a universal user base. Transition-probability idea retained; Q-learning replaced with real OULAD clickstream data.
[Reviewed paper 2]	[Method]	[Contribution]	[Narrow or platform-specific; failed the all-ages, all-levels filter]
[Reviewed paper 3]	[Method]	[Contribution]	[Limitation]

Table 1: Literature survey. Highlighted cells are to be completed from the papers reviewed so far.

Gap identified: the reviewed approaches are either trained on synthetic data, tied to a specific platform, or limited to prediction without feeding the result back into what the learner sees next. None combine real-data outcome prediction, learner clustering and a recommendation step in a single loop that works across ages and education levels. This is the specific gap the project targets.

3. Finalised System Architecture

The system follows a single-application layered design in which Flask sits between the user interface and two parallel services (storage and prediction) and merges their output into one result.

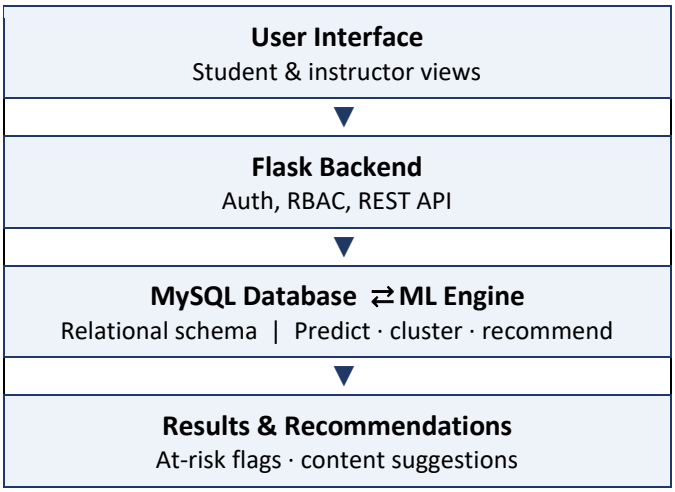


Fig. 1: System architecture (unchanged from Zeroth Review).

The adaptive loop runs continuously: the student logs in and starts a topic; behaviour is logged to MySQL; the ML model predicts current level and needs; the adaptive engine selects content; and the student's interaction generates new data that feeds the next prediction.

4. Module-Wise Design and Status

Module	Responsibility	Status
Database Layer	9-table MySQL schema: students, courses, content_items, student_interactions, assessments, student_scores, performance_predictions, learner_clusters, recommendations.	
Data Preparation	Exploration and cleaning of OULAD with pandas; loading cleaned data into MySQL.	
Classification Module	Decision Tree baseline followed by Random Forest to predict Pass / Fail / Distinction / Withdrawn; recall on Fail/Withdrawn weighted above raw accuracy.	
Clustering Module	Groups similar learners from activity and performance features.	
Flask Core	Authentication, role-based access, CRUD operations.	
Prediction API & Caching	Model served behind an API with a custom-built caching layer (reused from a prior project).	
Recommendation Engine	Converts prediction and cluster output into a content suggestion, using transition probabilities from real clickstream data.	

Table 2: Module breakdown against the architecture in Fig. 1.

5. Technology Stack (Finalised)

Layer	Tool / Framework	Purpose
Backend	Flask, Flask-SQLAlchemy, mysql-connector-python	Web application, ORM and database connectivity
Database	MySQL (9-table schema)	Relational storage of students, courses, scores and predictions
Machine learning	scikit-learn, pandas, numpy, joblib	Model training, data handling and model persistence
Auth & security	Flask-Login, Flask-WTF, werkzeug.security, python-dotenv	Sessions, form protection, password hashing, secrets
Report & testing	matplotlib, seaborn, pytest	Plots for evaluation and automated tests
Dataset & deployment	OULAD, gunicorn	Training data and production serving

Design decisions: Decision Tree is taught first so Random Forest is understood as many trees voting rather than a black box. A trained scikit-learn model runs free and offline with no API latency or outage risk. LLMs are limited to optional peripheral modules (quiz generation, teacher summaries, doubt-chatbot), isolated so their failure never affects the core prediction pipeline. Alternatives set aside: deep learning (data and compute needs), rule-based systems (not ML), collaborative filtering (needs large multi-user history), NoSQL (data is relational) and microservices (unneeded complexity for a single developer).

6. Implementation Progress So Far

- Development environment, repository and the 9-table MySQL schema: [confirm status].
- OULAD exploration and cleaning with pandas, and loading of cleaned data into MySQL: [confirm status].
- Decision Tree and Random Forest trained and compared; winning model saved with joblib: [confirm status and which model won].
- Flask core (authentication, roles, CRUD): [confirm status].
- Prediction API with caching and recommendation logic: [in progress / pending].

7. Preliminary Observations

[Add early findings here, for example: class distribution of the four outcome labels in OULAD, Decision Tree vs Random Forest recall on Fail/Withdrawn, and which activity features proved most predictive.] Full quantitative results are deferred to the evaluation phase once all components are integrated.

8. Challenges Encountered

- Mapping the OULAD tables onto the project's own 9-table relational schema without losing the student ↔ course ↔ score relationships. [confirm]
- Outcome classes are imbalanced, so recall on Fail and Withdrawn had to be prioritised over overall accuracy when comparing models. [confirm]

- Replacing the synthetic-data Q-learning approach in the assigned paper with transition probabilities derived from real clickstream data. [confirm]
- Keeping the design valid for all ages and education levels while training on a single institution's dataset. [confirm]

9. Revised Work Plan

Phase	Remaining Activity	Target
[#]	Complete model comparison; save winning model with joblib	[Weeks]
[#]	Build Flask core: authentication, roles, CRUD	[Weeks]
[#]	Integrate model behind prediction API, with caching	[Weeks]
[#]	Add clustering and recommendation logic	[Weeks]
[#]	End-to-end testing and deployment	[Weeks]
[#]	Consolidate results and write final report	[Weeks]

Table 3: Remaining phases from the original plan.

10. References

1. Kuzilek, J., Hlosta, M., & Zdrahal, Z. (2017). Open University Learning Analytics dataset. Scientific Data, 4, 170171.
2. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5–32.
3. [Assigned Q-learning recommender paper — full citation]
4. [Additional reviewed papers]
5. IEEE Xplore Digital Library. Source of the research papers reviewed for this project. [Online]. Available: https://ieeexplore.ieee.org/Xplore/home.jsp